27.3-2
Show that LFU has a competitive ratio of $\Theta(n/k)$ for the online caching problem with n requests and a cache of size k.

27.3-3
Show that FIFO has a competitive ratio of $O(k)$ for the online caching problem with n requests and a cache of size k.

27.3-4
Show that the deterministic MARKING algorithm has a competitive ratio of $O(k)$ for the online caching problem with n requests and a cache of size k.

27.3-5
Theorem 27.4 shows that any deterministic online algorithm for caching has a competitive ratio of $\Omega(k)$, where k is the cache size. One way in which an algorithm might be able to perform better is to have some ability to know what the next few requests will be. We say that an algorithm is ***l-lookahead*** if it has the ability to look ahead at the next l requests. Prove that for every constant $l \geq 0$ and every cache size $k \geq 1$, every deterministic l-lookahead algorithm has competitive ratio $\Omega(k)$.

Problems

27-1 Cow-path problem

The Appalachian Trail (AT) is a marked hiking trail in the eastern United States extending between Springer Mountain in Georgia and Mount Katahdin in Maine. The trail is about 2,190 miles long. You decide that you are going to hike the AT from Georgia to Maine and back. You plan to learn more about algorithms while on the trail, and so you bring along your copy of *Introduction to Algorithms* in your backpack.[2] You have already read through this chapter before starting out. Because the beauty of the trail distracts you, you forget about reading this book until you have reached Maine and hiked halfway back to Georgia. At that point, you decide that you have already seen the

trail and want to continue reading the rest of the book, starting with Chapter 28. Unfortunately, you find that the book is no longer in your pack. You must have left it somewhere along the trail, but you don't know where. It could be anywhere between Georgia and Maine. You want to find the book, but now that you have learned something about online algorithms, you want your algorithm for finding it to have a good competitive ratio. That is, no matter where the book is, if its distance from you is x miles away, you would like to be sure that you do not walk more than cx miles to find it, for some constant c. You do not know x, though you may assume that $x \geq 1$.[3]

What algorithm should you use, and what constant c can you prove bounds the total distance cx that you would have to walk? Your algorithm should work for a trail of any length, not just the 2,190-mile-long AT.

27-2 Online scheduling to minimize average completion time

Problem 15-2 discusses scheduling to minimize average completion time on one machine, without release times and preemption and with release times and preemption. Now you will develop an online algorithm for nonpreemptively scheduling a set of tasks with release times. Suppose you are given a set $S = \{a_1, a_2, \ldots, a_n\}$ of tasks, where task a_i has ***release time*** r_i, before which it cannot start, and requires p_i units of processing time to complete once it has started. You have one computer on which to run the tasks. Tasks cannot be ***preempted***, which is to say that once started, a task must run to completion without interruption. (See Problem 15-2 on page 446 for a more detailed description of this problem.) Given a schedule, let C_i be the ***completion time*** of task a_i, that is, the time at which task a_i completes processing. Your goal is to find a schedule that minimizes the average completion time, that is, to minimize $(1/n)\sum_{i=1}^{n} C_i$.

In the online version of this problem, you learn about task i only when it arrives at its release time r_i, and at that point, you know its processing time p_i. The offline version of this problem is NP-hard (see Chapter 34), but you will develop a 2-competitive online algorithm.

a. Show that, if there are release times, scheduling by shortest processing time (when the machine becomes idle, start the already released task with the smallest processing time that has not yet run) is not d-competitive for any constant d.

In order to develop an online algorithm, consider the preemptive version of this problem, which is discussed in Problem 15-2(b). One way to schedule is to run the tasks according to the shortest remaining processing time (SRPT) order. That is, at any point, the machine is running the available task with the smallest amount of remaining processing time.

b. Explain how to run SRPT as an online algorithm.

c. Suppose that you run SRPT and obtain completion times $C_1^P, \ldots, C_n^P$. Show that

$$\sum_{i=1}^{n} C_i^P \le \sum_{i=1}^{n} C_i^* ,$$

where the C_i^* are the completion times in an optimal nonpreemptive schedule.

Consider the (offline) algorithm COMPLETION-TIME-SCHEDULE.

COMPLETION-TIME-SCHEDULE(S)

1 compute an optimal schedule for the preemptive version of the problem
2 renumber the tasks so that the completion times in the optimal preemptive schedule are ordered by their completion times $C_1^P < C_2^P < \cdots < C_n^P$ in SRPT order
3 greedily schedule the tasks nonpreemptively in the renumbered order $a_1, \ldots, a_n$
4 let $C_1, \ldots, C_n$ be the completion times of renumbered tasks $a_1, \ldots, a_n$ in this nonpreemptive schedule
5 **return** $C_1, \ldots, C_n$

d. Prove that $C_i^P \geq \max\{\sum_{j=1}^{i} p_j, \max\{r_j : j \leq i\}\}$ for $i = 1, \ldots, n$.

e. Prove that $C_i \leq \max\{r_j : j \leq i\} + \sum_{j=1}^{i} p_j$ for $i = 1, \ldots, n$.

f. Algorithm COMPLETION-TIME-SCHEDULE is an offline algorithm. Explain how to modify it to produce an online algorithm.

g. Combine parts (c)–(f) to show that the online version of COMPLETION-TIME-SCHEDULE is 2-competitive.

Chapter notes

Online algorithms are widely used in many domains. Some good overviews include the textbook by Borodin and El-Yaniv [68], the collection of surveys edited by Fiat and Woeginger [142], and the survey by Albers [14].

The move-to-front heuristic from Section 27.2 was analyzed by Sleator and Tarjan [416, 417] as part of their early work on amortized analysis. This rule works quite well in practice.

Competitive analysis of online caching also originated with Sleator and Tarjan [417]. The randomized marking algorithm was proposed and analyzed by Fiat et al. [141]. Young [464] surveys online caching and paging algorithms, and Buchbinder and Naor [76] survey primal-dual online algorithms.

Specific types of online algorithms are described using other names. ***Dynamic graph algorithms*** are online algorithms on graphs, where at each step a vertex or edge undergoes modification. Typically a vertex or edge is either inserted or deleted, or some associated property, such as edge weight, changes. Some graph problems need to be solved again after each change to the graph, and a good dynamic graph algorithm will not need to solve from scratch. For example, edges are inserted and deleted, and after each change to the graph, the minimum spanning tree is recomputed. Exercise 21.2-8 asks such a question. Similar questions can be asked for other graph algorithms, such as shortest paths, connectivity, or matching. The first paper in this field is credited to Even and Shiloach [138], who study how to maintain a shortest-path

tree as edges are being deleted from a graph. Since then hundreds of papers have been published. Demetrescu et al. [110] survey early developments in dynamic graph algorithms.

For massive data sets, the input data might be too large to store. ***Streaming algorithms*** model this situation by requiring the memory used by an algorithm to be significantly smaller than the input size. For example, you may have a graph with n vertices and m edges with $m \gg n$, but the memory allowed may be only $O(n)$. Or you may have n numbers, but the memory allowed may only be $O(\lg n)$ or $O(\sqrt{n})$. A streaming algorithm is measured by the number of passes made over the data in addition to the running time of the algorithm. McGregor [322] surveys streaming algorithms for graphs and Muthukrishnan [341] surveys general streaming algorithms.

---

[1] The path-compression heuristic in Section 19.3 resembles MOVE-TO-FRONT, although it would be more accurately expressed as "move-to-next-to-front." Unlike MOVE-TO-FRONT in a doubly linked list, path compression can relocate multiple elements to become "next-to-front."

[2] This book is heavy. We do not recommend that you carry it on a long hike.

[3] In case you're wondering what this problem has to do with cows, some papers about it frame the problem as a cow looking for a field in which to graze.